Informe de Seguridad

Sitio web de pedidos "Tango Fast Food"

https://valengarcia09.github.io/guesaa/ · 23 de septiembre de 2026

1. Alcance y metodología

Se realizó una revisión pasiva del código público del sitio (HTML y script.js), tal como lo descarga cualquier visitante. No se realizaron ataques, pruebas intrusivas ni envíos de pedidos reales.

El sitio es estático (alojado en GitHub Pages): no tiene servidor propio ni base de datos. Toda la lógica corre en el navegador del cliente y el pedido se envía por WhatsApp. Por este motivo no aplican vulnerabilidades clásicas de servidor (inyección SQL, robo de sesiones, etc.); los riesgos se concentran en la lógica de negocio y en el código del lado del cliente.

2. Resumen de hallazgos

Se detectaron **9 hallazgos**: 3 de severidad alta, 2 de severidad media y 4 de severidad baja. Los de severidad alta afectan directamente al negocio (dinero y reputación) y deberían atenderse primero.

N°	Vulnerabilidad	Severidad
1	Manipulación de precios desde el navegador	**Alta**
2	Falta de verificación de pagos y pedidos	**Alta**
3	Riesgo de suplantación o clonado del sitio (phishing)	**Alta**
4	Inyección de HTML / Self-XSS en el resumen del pedido	**Media**
5	Exposición de datos personales	**Media**
6	Ausencia de cabeceras de seguridad (CSP, anti-clickjacking)	**Baja**
7	Enlaces a pestañas nuevas sin rel="noopener"	**Baja**
8	Horario de atención controlado por el reloj del cliente	**Baja**
9	Recursos de terceros y privacidad	**Baja**

Criterio de severidad: Alta = puede causar pérdida de dinero o fraude a clientes. Media = expone datos o abre la puerta a ataques si el sitio crece. Baja = buena práctica recomendada, con impacto actual limitado.

3. Detalle de vulnerabilidades y recomendaciones

3.1 Manipulación de precios desde el navegador

Severidad: **Alta**

Descripción: Los precios de los productos se leen del propio HTML (atributos value de los <select> y parámetros de funciones como agregarDirecto('...', 8000)) y el total se calcula íntegramente en el navegador del cliente.

Evidencia:

```
<option value="9000" data-nombre="Simple">Simple - $9.000</option>
carrito[0].precio = 1;   // ejecutado en la consola (F12)
```

Impacto: Cualquier persona puede modificar los precios con las herramientas de desarrollador y enviar por WhatsApp un pedido con un "TOTAL FINAL" falso. El botón de Mercado Pago también muestra el monto alterado. Si el personal confía en el total del mensaje, el local cobra de menos.

Recomendación:

- Nunca confiar en el total que llega por WhatsApp: recalcularlo del lado del local con la lista de precios oficial.
- Idealmente, que el mensaje incluya solo productos, variantes y cantidades (sin precios), o que el local tenga una planilla o sistema que calcule el total.
- A mediano plazo, usar un backend o una plataforma de pedidos donde el precio se valide en el servidor.

3.2 Falta de verificación de pagos y pedidos

Severidad: **Alta**

Descripción: El flujo de pago consiste en transferir a un alias y enviar el comprobante por chat. No hay confirmación automática del pago ni validación de la identidad o dirección del cliente.

Impacto: Es sencillo enviar comprobantes de transferencia falsificados (editados o generados). También se pueden enviar pedidos falsos o de broma con direcciones inventadas, sin ningún límite.

Recomendación:

- Verificar cada pago directamente en la app de Mercado Pago antes de preparar o despachar el pedido; no alcanza con la captura de pantalla.
- Usar links de pago de Mercado Pago con monto fijo o Checkout Pro, que confirman el pago automáticamente.
- Para pedidos en efectivo, confirmar por llamada o mensaje en pedidos de clientes nuevos o montos altos.

3.3 Riesgo de suplantación o clonado del sitio (phishing)

Severidad: **Alta**

Descripción: El sitio se publica en un subdominio gratuito (valengarcia09.github.io/guesaa) y el repositorio de código es público.

Impacto: Un atacante puede copiar el sitio en minutos, reemplazar el alias de pago y el número de WhatsApp por los suyos y difundirlo como si fuera el oficial. Los clientes no tienen una forma sencilla de distinguir el sitio verdadero del falso.

Recomendación:

- Registrar un dominio propio (por ejemplo, tangofastfood.com.ar) y apuntarlo a GitHub Pages.
- Publicar el link oficial en la bio de Instagram y en el perfil de WhatsApp Business.
- Avisar a los clientes cuál es el único alias válido para pagar.

3.4 Inyección de HTML / Self-XSS en el resumen del pedido

Severidad: **Media**

Descripción: En la función enviarWhatsApp() los campos Nombre, Dirección, Entre calles y Notas se insertan en el modal de resumen mediante innerHTML sin escapar.

Evidencia:

```
resumenHTML += `<p><b>Notas:</b> ${observaciones}</p>`;
document.getElementById("contenido-modal").innerHTML = resumenHTML;

// Escribir en "Notas":  <img src=x onerror=alert(1)>   -> ejecuta JavaScript
```

Impacto: Hoy el impacto es limitado porque solo afecta a quien escribe el texto (no se guarda ni viaja en la URL). Pero si en el futuro los pedidos se almacenan o se muestran en un panel del local, se convierte en un XSS almacenado capaz de ejecutar código en el equipo del personal. Lo mismo ocurre en actualizarCarrito() con los nombres de productos.

Recomendación:

- Escapar el texto antes de insertarlo con innerHTML, o construir los elementos con textContent:

```
function escaparHTML(texto) {
  return String(texto)
    .replace(/&/g, "&amp;").replace(/</g, "&lt;")
    .replace(/>/g, "&gt;").replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}

// Uso:
resumenHTML += `<p><b>Notas:</b> ${escaparHTML(observaciones)}</p>`;
```

3.5 Exposición de datos personales

Severidad: **Media**

Descripción: El nombre completo del titular de la cuenta, el alias de Mercado Pago y el número de teléfono están escritos en texto plano en el HTML y en el JavaScript.

Impacto: Estos datos pueden recolectarse automáticamente para spam, intentos de estafa o ingeniería social (por ejemplo, mensajes falsos haciéndose pasar por Mercado Pago o por el banco).

Recomendación:

- Evaluar si es necesario publicar el nombre completo del titular; mostrarlo solo al momento de confirmar un pedido por transferencia.
- Usar una cuenta y un número de WhatsApp Business dedicados exclusivamente al local, separados de los personales.

3.6 Ausencia de cabeceras de seguridad (CSP, anti-clickjacking)

Severidad: **Baja**

Descripción: El sitio no define Content-Security-Policy, X-Frame-Options ni frame-ancestors. GitHub Pages no permite configurar cabeceras HTTP personalizadas.

Impacto: Otro sitio puede mostrar la página dentro de un iframe (clickjacking) y, sin CSP, cualquier script inyectado se ejecutaría sin restricciones.

Recomendación:

- Agregar una CSP mediante etiqueta meta (no protege contra iframes, pero limita los scripts):

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; img-src 'self' data:;
               style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
               font-src https://fonts.gstatic.com; script-src 'self'">
```

Nota: como el sitio usa atributos onclick en línea, para aplicar esta CSP estricta hay que mover esos eventos a addEventListener dentro de script.js. Para protección completa contra iframes se puede poner un proxy como Cloudflare (plan gratuito) delante de un dominio propio.

3.7 Enlaces a pestañas nuevas sin rel="noopener"

Severidad: Baja

Descripción: window.open(url, '_blank') al enviar el pedido y el botón de Mercado Pago (target="_blank") no usan noopener.

Impacto: La pestaña abierta obtiene una referencia a la página original (reverse tabnabbing). El riesgo es bajo porque los destinos son sitios confiables y los navegadores modernos lo mitigan por defecto.

Recomendación:

- Agregar noopener en ambos casos:

```
window.open(url, "_blank", "noopener,noreferrer");

<a id="btn-pago-mp" href="..." target="_blank" rel="noopener noreferrer">
```

3.8 Horario de atención controlado por el reloj del cliente

Severidad: Baja

Descripción: La función verificarEstadoLocal() usa la fecha y hora del dispositivo del visitante para mostrar "Local abierto/cerrado", y el envío de pedidos no se bloquea fuera de horario.

Impacto: Cambiando la hora del equipo, el sitio muestra "abierto"; además, se pueden enviar pedidos en cualquier momento. No es grave, pero la regla de negocio no se aplica realmente.

Recomendación:

- Deshabilitar el botón de envío fuera de horario y, sobre todo, que el local ignore o responda los pedidos recibidos fuera de horario.
- Calcular la hora en la zona horaria de Argentina (America/Argentina/Buenos_Aires) para no depender de la configuración del visitante.

3.9 Recursos de terceros y privacidad

Severidad: Baja

Descripción: Las fuentes tipográficas se cargan desde Google Fonts.

Impacto: Google recibe la dirección IP y datos de navegación de cada visitante. Es un tema menor de privacidad.

Recomendación:

- Descargar las fuentes y alojarlas en el mismo repositorio (self-hosting).

4. Aspectos positivos

- HTTPS forzado con HSTS (provisto por GitHub Pages).
- No se encontraron claves de API, contraseñas ni secretos en el código.
- No están expuestos archivos sensibles como .git/config o .env (responden 404).
- Los enlaces a redes sociales ya utilizan rel="noopener noreferrer".

5. Plan de acción sugerido

Inmediato (sin programar)

- Verificar cada pago en la app de Mercado Pago y recalcular el total de cada pedido con la lista de precios oficial.
- Publicar el link oficial del sitio y el alias válido en Instagram y WhatsApp Business.

Corto plazo (cambios chicos de código)

- Escapar los datos del cliente antes de usar innerHTML (punto 3.4).
- Agregar noopener a los enlaces y a window.open (punto 3.7).
- Agregar una Content-Security-Policy mediante meta tag (punto 3.6).
- Revisar el historial de commits del repositorio público por si alguna vez se subió información sensible.

Mediano plazo

- Dominio propio para el sitio.
- Integración con Checkout de Mercado Pago o un backend que valide precios y confirme pagos automáticamente.